

BuildTrack — Complete Platform Specification & Implementation Map

All-in-one canvas: full end-to-end design, diagrams, workflows, APIs, DB schema, infra, frontend/backend folder layouts, and implementation checklist. This document collects **everything** we discussed (from first to last message) and expands every feature so you can hand it to developers or share with friends. Read top-to-bottom — it's intentionally exhaustive.

Quick navigation

- Executive summary
 - Roles & entities
 - Exhaustive feature list (final)
 - High-level architecture (diagram + explanation)
 - Workflows (detailed step-by-step nodes with checks)
 - API surface (grouped endpoints + brief contract)
 - Database schema (full SQL tables + constraints)
 - Backend services & components (what code does what)
 - Frontend screens & component map
 - Folder structure (backend + frontend with file names)
 - Important implementation notes (transactions, concurrency, security)
 - Operational concerns (scaling, background jobs, monitoring)
 - UX / visual design notes and sample wireframes
 - Roadmap & prioritized tasks
 - Appendix: code snippets for critical flows
-

1. Executive summary

BuildTrack connects parents, children (students), and project builders to help students complete school projects. Parents find and hire builders (or buy ready-made projects), pay securely (only parents), schedule sessions respecting children availability, and track progress. Builders teach online, deliver offline (in-person or courier), or provide hybrid services. System supports real-time notifications, media-rich portfolios (S3 + thumbnails), catalog buying, refunds/cancellations, and admin tools. Family-centric model ensures multiple parents share the same family data while preventing duplicate payments.

Goal of this document: provide a single, shareable specification with architecture diagrams, workflows, APIs, DB schema, and a file/folder structure to implement the platform end-to-end.

2. Roles & primary entities

- **Family** — household container: many parents, many children. Core ownership unit.
 - **Parent** — adult user; can manage children, approve services, pay, request refunds, set availability.
 - **Child** — student user; sees only services assigned to them; cannot pay or start services.
 - **Builder** — service provider; lists services and ready-made projects; uploads media; updates progress.
 - **Service** — learning offering by a builder (price, duration, delivery options).
 - **BuilderProject** — ready-made model in the builder catalog (price + delivery options).
 - **Subscription / ChildService** — booking linking family → child → service, with statuses (pending, approved, active, completed, cancelled, refunded).
 - **Payment** — parent payment record for subscription or project order.
 - **Notification** — persisted alerts, pushed in real-time.
 - **Media** — images/videos attached to builders' services/projects (stored in S3 + thumbnails).
-

3. Exhaustive feature list (final)

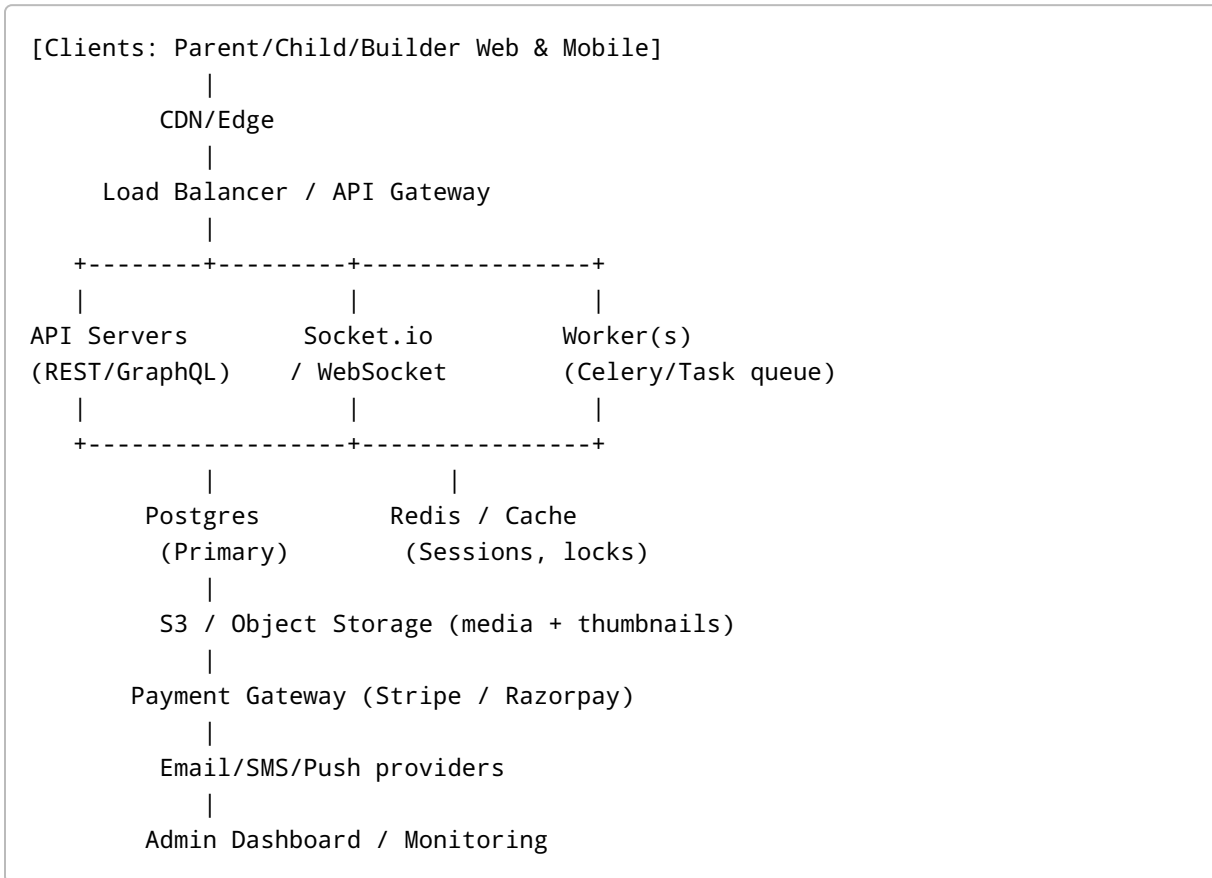
Parent features - Family account: invite parent, join family. Single family data for multiple parents. - Create & manage child profiles (name, grade, age, avatar). - Set child availability (weekly recurring slots: day of week + time ranges). - Browse/search/filter builders (expertise, rating, cost slider, location, delivery mode). - Compare up to 3 builders side-by-side (expertise, rating, price, portfolio). - Book learning services (booked → pay → active → complete). - Payment & checkout UI; receipts & invoice download. - Refund & cancel flows (cancel only before service started; refund with rules). - Opt-in model add-on after learning (pay extra if parent wants finished model delivered). - Direct project purchase from builder catalog. - Notification center + real-time push; unread badge; history & mark-read/clear. - View project progress timeline & milestones.

Child features - Dashboard (only assigned services visible). - Simple progress UI and session reminders. - Notifications relevant to child. - No payments and no builder selection rights.

Builder features - Onboard & verify (KYC optional), profile, expertise, location, avatar. - Define services (price, duration, delivery-mode: online / offline / hybrid), booking expiry days. - Upload media (images, videos) — S3 storage + thumbnails for images. - Catalog listing for ready projects (price + delivery options: courier, in-person, both). - Scheduling: propose slots aligned with child availability; confirm sessions. - Update progress (milestones, percent, optional media) — creates timeline entries + notifications. - Receive notifications (new booking, payment, cancellation, refund). - Earnings dashboard, payouts, subscription tiers (free/premium).

Platform/Admin features - Admin panel for KYC, dispute resolution, refunds, analytics. - Escrow option (hold funds until service start); release to builder on start or completion. - Background jobs: expiry checker, invoice generator, analytics jobs. - Logging & audit trail for transactions & notifications.

4. High-level architecture



Explanation: - Clients connect to API servers for CRUD operations and to Socket.IO server for real-time notifications. - Postgres stores relational data (families, parents, children, services, subscriptions, payments, notifications, media). - Redis is used for caching, session storage, and distributed locks. - S3 stores media and thumbnails; presigned uploads for security and scalability. - Worker fleet (Celery/RQ) runs background tasks: thumbnail generation, expiry checker, invoice generation, scheduled reminders. - Payments processed via third-party gateway; consider escrow/account transfers for builder payouts.

5. Detailed Workflows (step-by-step nodes)

Each workflow below is written as steps and includes DB checks / notification points / failure cases.

A. Booking / Subscription Workflow (Child request → Parent pay)

1. **Child requests service** (UI: "Request builder X for service Y")
2. API: POST /children/{child_id}/services/{service_id}/request
3. Action: Create subscription row with status = pending, payment_status = unpaid, include family_id.

4. Notify family (all parents) via WebSocket: `child_requested`.

5. Parent reviews request

6. Parent opens Notification Center or Family Dashboard.

7. Parent can `Approve & Pay` (start payment flow) or `Reject`.

8. Atomic approve + payment (key point)

9. API: `POST /parents/{parent_id}/subscriptions/{subscription_id}/approve` triggers transaction.

10. Implementation detail (transaction-safe): run `SELECT ... FOR UPDATE` on the subscription (or on uniqueness slot), check `status` still `pending` and `payment_status` `unpaid`.

11. If ok, call payment gateway (charge card). On success:

- Update `subscription.status = approved/active`, `payment_status = paid`, `start_date`, `end_date`.
- If using escrow, move funds to escrow.
- Notify family & builder (`subscription_approved`).

12. If payment fails: return error; notify parent.

13. If another parent already approved: return `409 Conflict` and notify requester.

14. Service starts only after payment (and optionally scheduled start)

15. For online sessions: builder must propose a slot matching child availability and parent must confirm the slot; start only after both conditions satisfied.

16. For offline/hybrid: after payment builder either schedules delivery or marks as delivering.

17. During service

18. Builder posts milestones updates → create progress entries and notifications.

19. Parents & child view timeline and progress bar.

20. Service completion

21. Builder marks completed → `subscription.status = completed`.

22. Parent can optionally opt-in to buy model (if hybrid) — triggers separate payment & delivery.

B. Payments, Refunds & Cancel Flow (rules & step nodes)

Rules: - Cancel without payment: allowed anytime while `status = booked`. - Cancel after payment: allowed only before service start and if policy allows (system may allow refunds until builder started work). If allowed, refund processed via gateway and `payment_status = refunded`. - Refunds require DB update and notifications to both parent & builder.

APIs & steps - `POST /api/assign-service` creates `child_services` and associated `payments` row with `status = pending`. - `POST /api/pay` processes payment: check `payment.status == pending`, charge gateway, mark `paid`, update subscription to `active`, notify builder. - `POST /api/cancel` (parent cancels): allowed only if `subscription.status == booked` OR `payment.pending`. If cancelled, change `subscription.status = cancelled`, delete payment row or mark `cancelled` and notify builder/parent. - `POST /api/refund` (parent requests refund): allowed if `payment.status == paid` and business rules allow. On success: `payment.status = refunded`, `subscription.status = cancelled` and notify parties.

Atomicity precautions: - Use DB transactions with `FOR UPDATE` on subscription/payment rows. - Use unique constraints on `(family_id, child_id, service_id)`. - Use Redis locks for distributed environments as defense-in-depth.

C. Media upload (S3 + thumbnails)

1. Builder chooses files in UI (images/videos).
2. Frontend requests a presigned POST/PUT URL from backend: `GET /api/media/presign?filename=...&contentType=...`.
3. Frontend uploads directly to S3 using presigned URL (multipart/form-data) to avoid sending large files through backend.
4. Backend creates `media` DB record only after confirming upload or receives upload callback (or frontend notifies backend with uploaded filename).
5. Worker (Celery) picks image uploads and generates thumbnails (Pillow), stores thumbnail in S3, updates `media.thumbnail_url`.
6. Frontend gallery requests `project` and gets `media` array with `url` and `thumbnail` to show gallery.

Limits & validation - Video limit: 50MB per file enforced at frontend and via presigned policy. Block unacceptable mime-types. - Image limit: e.g., 10MB.

D. Search, Filter & Compare builders

- Endpoint:
`GET /api/builders?expertise=&location=&minRating=&minCost=&maxCost=&q=`.
- Server returns paginated results sorted by relevance or sponsored (builders with subscription tier get priority).
- Compare UI: client side holds up to 3 selected builder IDs and shows columns for expertise, rating, cost, portfolio, number of projects, response time.

E. Real-time notifications

- Use Socket.IO (Flask-SocketIO or Node socket.io) or WebSockets (FastAPI/natively).
- Users join `family_{family_id}` room so both parents + children receive family-level notifications.
- Builders join `builder_{builder_id}` room.
- Notification flow uses `notify_and_save(family_id, payload)` which:

- Writes row in `notifications` table for history.
- Emits payload to WebSocket room.
- API provides `GET /families/{id}/notifications` and `POST /notifications/{id}/read` and `GET /families/{id}/notifications/unread_count`.

F. Scheduling & Availability

- **Parent sets availability** (recurring slots). Table: `child_availability(child_id, day_of_week, start_time, end_time)`.
- **Builder proposes slots:** `POST /api/subscriptions/{id}/propose_slot` with timestamp(s). Backend verifies the slot falls within child availability.
- **Parent confirms slot:** `POST /api/subscriptions/{id}/confirm_slot` → schedulers set `start_time` and `status='active'`.
- Send auto-reminders via worker (email / push) 24 hours and 1 hour before session.

G. Hybrid / Offline / Model add-on flow

- When parent books a hybrid-enabled service, default flow is **teaching**.
- After teaching completes (status `completed_online`) parent is prompted: "Buy finished model? Price ₹X".
- If parent opts in: create a `model_payment` record; on payment success, update `child_services.model_status='model_in_transit'`, create `delivery_tracking`, notify builder to ship.
- Direct purchase path: `POST /api/buy_project` (for catalog items) creates `project_orders` and payment row.

H. Expiry & background jobs

- Each builder sets `booking_expiry_days` for their services.
- Worker job `expire_bookings` runs every 30 minutes:
- `UPDATE child_services SET status='expired' WHERE status='booked' AND expires_at < now()`
- Notify parents & builder.

6. API surface (grouped)

Note: auth middleware returns `user_id`, `role`, `family_id` in request context. Use JWT for auth.

Auth

- `POST /auth/signup` — create parent / builder / child (invites).
- `POST /auth/login` — return JWT with `{ user_id, role, family_id }`.
- `POST /auth/invite-parent` — invite second parent to family.

Family & Users

- GET /families/{family_id} — family summary.
- GET /parents/{id} — parent profile.
- GET /children/{id} — child profile & assigned services.
- POST /children — add child to family.
- PUT /children/{id}/availability — set availability.

Builders & Search

- GET /builders — search & filter.
- GET /builders/{id} — builder profile with media, rating, services, catalog.
- POST /builders/{id}/media/presign — get presigned URL.
- POST /builders/{id}/services — create service (builder-only).
- GET /builders/{id}/projects — catalog list.

Subscriptions / Bookings

- POST /children/{child_id}/services/{service_id}/request — child requests.
- GET /families/{family_id}/subscriptions — family subscriptions list.
- POST /parents/{parent_id}/subscriptions/{subscription_id}/approve — approve + pay (atomic).
- POST /subscriptions/{id}/propose_slot — builder proposes slots.
- POST /subscriptions/{id}/confirm_slot — parent confirms slot.
- POST /subscriptions/{id}/start — builder starts service (checks active/payed & availability).
- POST /subscriptions/{id}/complete — builder marks completed.
- POST /subscriptions/{id}/opt_model — parent opts to buy model (post-service add-on).

Payments & Orders

- POST /payments/create — create payment intent / record (links to subscription or project_order).
- POST /payments/confirm — confirm on webhook/callback from gateway.
- POST /payments/refund — initiate refund (admin or parent request).
- POST /project_orders — parent purchases catalog item.
- PUT /project_orders/{id}/status — update order status.

Media & Gallery

- POST /media/presign — presigned URL.
- GET /projects/{project_id}/media — list media.

Notifications

- GET /families/{id}/notifications — list notifications (history).
- GET /families/{id}/notifications/unread_count — count unread.

- `POST /notifications/{id}/read` — mark read.
- WebSocket endpoint: `ws://.../ws` — join family room: `join_family` with `{family_id}`.

Admin

- `GET /admin/pending-kyc` — list builders waiting verification.
- `POST /admin/resolve-dispute` — handle disputes.

7. Database schema (full SQL)

Below are the main tables (Postgres). Use migrations (Alembic) in production.

```
-- families
CREATE TABLE families (
    id SERIAL PRIMARY KEY,
    name TEXT,
    created_at TIMESTAMP DEFAULT now()
);

-- parents
CREATE TABLE parents (
    id SERIAL PRIMARY KEY,
    family_id INT REFERENCES families(id) ON DELETE CASCADE,
    name TEXT,
    email TEXT UNIQUE NOT NULL,
    password_hash TEXT NOT NULL,
    role TEXT DEFAULT 'parent',
    created_at TIMESTAMP DEFAULT now()
);

-- children
CREATE TABLE children (
    id SERIAL PRIMARY KEY,
    family_id INT REFERENCES families(id) ON DELETE CASCADE,
    name TEXT,
    grade TEXT,
    age INT,
    created_at TIMESTAMP DEFAULT now()
);

-- builders
CREATE TABLE builders (
    id SERIAL PRIMARY KEY,
    name TEXT,
```

```

    city TEXT,
    expertise TEXT,
    kyc_status TEXT DEFAULT 'pending',
    booking_expiry_days INT DEFAULT 3,
    subscription_tier TEXT DEFAULT 'free',
    created_at TIMESTAMP DEFAULT now()
);

-- services (by builders)
CREATE TABLE services (
    id SERIAL PRIMARY KEY,
    builder_id INT REFERENCES builders(id),
    title TEXT,
    description TEXT,
    price NUMERIC(10,2),
    duration_days INT,
    delivery_modes TEXT, -- e.g. 'online,offline,hybrid'
    created_at TIMESTAMP DEFAULT now()
);

-- builder_projects (catalog)
CREATE TABLE builder_projects (
    id SERIAL PRIMARY KEY,
    builder_id INT REFERENCES builders(id),
    title TEXT,
    description TEXT,
    price NUMERIC(10,2),
    delivery_option TEXT, -- courier, in_person, both
    estimated_delivery_days INT DEFAULT 7,
    created_at TIMESTAMP DEFAULT now()
);

-- subscriptions (family -> child -> service)
CREATE TABLE subscriptions (
    id SERIAL PRIMARY KEY,
    family_id INT REFERENCES families(id) ON DELETE CASCADE,
    child_id INT REFERENCES children(id) ON DELETE CASCADE,
    service_id INT REFERENCES services(id) ON DELETE CASCADE,
    status TEXT CHECK (status IN
('pending','booked','approved','active','completed','cancelled','expired','refunded'))
    DEFAULT 'pending',
    payment_status TEXT CHECK (payment_status IN
('unpaid','pending','paid','refunded')) DEFAULT 'unpaid',
    start_date TIMESTAMP,
    end_date TIMESTAMP,
    model_opted BOOLEAN DEFAULT FALSE,
    model_price NUMERIC(10,2),
    model_status TEXT, -- model_pending_payment, model_in_transit, delivered

```

```

    delivery_mode TEXT, -- online, offline, hybrid
    expires_at TIMESTAMP,
    created_at TIMESTAMP DEFAULT now(),
    updated_at TIMESTAMP DEFAULT now(),
    UNIQUE (family_id, child_id, service_id)
);

-- payments (link to subscription or project_order)
CREATE TABLE payments (
    id SERIAL PRIMARY KEY,
    parent_id INT REFERENCES parents(id),
    subscription_id INT REFERENCES subscriptions(id),
    project_order_id INT REFERENCES project_orders(id),
    amount NUMERIC(10,2),
    status TEXT CHECK (status IN
('pending', 'paid', 'failed', 'refunded', 'cancelled')) DEFAULT 'pending',
    transaction_id TEXT,
    refund_id TEXT,
    created_at TIMESTAMP DEFAULT now(),
    paid_at TIMESTAMP
);

-- project_orders
CREATE TABLE project_orders (
    id SERIAL PRIMARY KEY,
    parent_id INT REFERENCES parents(id),
    builder_id INT REFERENCES builders(id),
    project_id INT REFERENCES builder_projects(id),
    child_id INT REFERENCES children(id),
    order_status TEXT CHECK (order_status IN
('pending_payment', 'paid', 'in_transit', 'delivered', 'cancelled')) DEFAULT
'pending_payment',
    delivery_tracking TEXT,
    created_at TIMESTAMP DEFAULT now()
);

-- media
CREATE TABLE media (
    id SERIAL PRIMARY KEY,
    builder_id INT REFERENCES builders(id),
    service_id INT REFERENCES services(id),
    project_id INT REFERENCES builder_projects(id),
    url TEXT NOT NULL,
    thumbnail_url TEXT,
    media_type TEXT CHECK (media_type IN ('image', 'video')),
    created_at TIMESTAMP DEFAULT now()
);

```

```

-- notifications (family-level and user-level)
CREATE TABLE notifications (
  id SERIAL PRIMARY KEY,
  family_id INT REFERENCES families(id),
  user_id INT, -- nullable: for cross-user notifications
  type TEXT,
  message TEXT NOT NULL,
  data JSONB,
  read BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP DEFAULT now()
);

-- child_availability
CREATE TABLE child_availability (
  id SERIAL PRIMARY KEY,
  child_id INT REFERENCES children(id),
  day_of_week TEXT, -- Monday..Sunday
  start_time TIME,
  end_time TIME
);

-- audit logs
CREATE TABLE audit_logs (
  id SERIAL PRIMARY KEY,
  actor_id INT,
  actor_role TEXT,
  action TEXT,
  details JSONB,
  created_at TIMESTAMP DEFAULT now()
);

```

8. Backend architecture & services

Break backend into modular services (monolith or microservices). Suggested microservice boundaries for clarity:

1. **Auth Service** (JWT, password hashing, invite links)
2. **User Service** (parents, children, family management)
3. **Builder Service** (builder profiles, KYC, services & catalog)
4. **Subscription Service** (bookings, scheduling, lifecycle)
5. **Payment Service** (integration with payment gateway + ledger + refunds)
6. **Media Service** (presign, store metadata, thumbnailing worker)
7. **Notification Service** (Socket.IO + notification persistence)
8. **Worker / Background** (Celery / RQ for thumbnailing, expiry, reminders)

9. Admin Service (KYC review, disputes, analytics)

Inter-service communication - Use REST or gRPC between services or keep services as modules in a single monolith for v1. - Use message broker (RabbitMQ / Redis streams / Kafka) for events: `subscription_approved`, `payment_completed`, `refund_processed`, `media_uploaded`, `booking_expired`.

Key libraries - Python: FastAPI or Flask, SQLAlchemy, Alembic, Celery, boto3 - Node: Express + socket.io (alternative stack) - DB: PostgreSQL, Redis - Storage: AWS S3 - Payments: Stripe or Razorpay (or simulated for PoC)

9. Frontend architecture & page/component map

Use React (Next.js recommended) for universal rendering and SEO. State management: React Query / SWR for server data, context for auth, and a small local store for compare selections.

Pages / Routes - `/` — landing page (hero, search, featured builders) - `/auth/login`, `/auth/signup`, `/auth/invite` — authentication - `/dashboard` — family dashboard (parents); child dashboard when child logs in - `/builders` — directory (search + filters) - `/builders/[id]` — builder profile & gallery - `/compare` — comparison page - `/projects/catalog` — builder project store - `/project/[id]` — catalog project details + buy - `/subscriptions` — family subscriptions list - `/subscriptions/[id]` — subscription detail + timeline - `/payments` — pending payments & history - `/notifications` — notification center - `/admin` — admin dashboard (KYC / disputes)

Important components - `NavBar` (role-aware) - `Notifications` (bell + dropdown + center) - `BuilderCard`, `BuilderProfile`, `MediaGallery` (grid + thumbnails) - `MediaUploader` (presign + preview) - `CompareBar` (floating) - `AssignServiceModal` (choose child + service) - `PaymentModal` (checkout simulation) - `ProgressTimeline` (list of milestones) - `Scheduler` (propose & confirm slots)

10. Folder structure & key files

Below are **suggested** structures. Keep them tidy and modular so teams can collaborate.

Backend (Python / FastAPI + SQLAlchemy)

```
backend/
├── services/
│   ├── auth/
│   │   ├── __init__.py
│   │   ├── routes.py
│   │   └── models.py
```

```

| |   └ schemas.py
| |   └ users/
| |     └ routes.py
| |     └ models.py
| |     └ crud.py
| |   └ builders/
| |     └ routes.py
| |     └ models.py
| |     └ crud.py
| |     └ media_utils.py
| |   └ subscriptions/
| |     └ routes.py
| |     └ models.py
| |     └ booking_logic.py
| |   └ payments/
| |     └ routes.py
| |     └ gateway.py
| |     └ ledger.py
| |   └ notifications/
| |     └ routes.py
| |     └ socketio_handler.py
| |     └ service.py
| |   └ admin/
| |     └ routes.py
| |     └ kyc.py
| └ workers/
|   └ thumbnail_worker.py
|   └ expiry_checker.py
|   └ reminders.py
└ migrations/ # alembic
└ tests/
└ alembic.ini
└ Dockerfile
└ docker-compose.yml
└ main.py # launches FastAPI + SocketIO

```

Frontend (Next.js / React + Tailwind + shadcn/ui)

```

frontend/
└ app/ # Next.js pages or src/pages
  └ page.tsx
  └ dashboard/
    └ page.tsx
    └ components/
  └ builders/

```

```

| | | index.tsx
| | | └ [id].tsx
| | notifications/
| | | page.tsx
| | └ subscriptions/
└ components/
  | NavBar.tsx
  | Notifications.tsx
  | BuilderCard.tsx
  | MediaUploader.tsx
  | MediaGallery.tsx
  | ProgressTimeline.tsx
  | └ Scheduler.tsx
└ hooks/
  | useSocket.ts
  | useAuth.ts
  | └ useNotifications.ts
└ lib/
  | api.ts
  | └ formatter.ts
└ styles/
  | └ globals.css
└ public/
└ next.config.js
└ package.json
└ tailwind.config.js

```

Notes on files - `subscriptions/booking_logic.py` contains the `SELECT ... FOR UPDATE` transaction code for approval. - `notifications/socketio_handler.py` is the server WebSocket glue. - `workers/thumbnail_worker.py` uses Pillow to generate thumbnails and update the DB. - Keep `media_utils.py` for S3 presign and validation logic.

11. Security, Data privacy & Access control

Auth & Sessions - Use JWT with short expiry + refresh tokens. - Secure password hashing (bcrypt/argon2). - Use scopes in JWT: role (parent/child/builder/admin), family_id.

Authorization rules - Child endpoints only accessible if token role is `child` and `child_id` matches token. - Parent endpoints enforce `family_id` match; only parents of the family can access family resources. - Builder endpoints require `builder_id` match. - Admin endpoints require `role=admin` and additional 2FA for sensitive flows.

Payments - PCI compliance: use hosted pages or tokenized gateways (Stripe Elements / Razorpay Checkout) to avoid handling card data. - Use idempotency keys for payment endpoints to avoid duplicate charges on retries.

Media - Use presigned S3 uploads with limited expiry and file size / content-type policy. - Validate file headers (magic bytes) on worker for extra safety.

Rate-limits & Abuse detection - Rate limit critical endpoints (login, payment) with IP-based throttling. - Monitor suspicious activity (multiple failed payment attempts).

Privacy - Children data is sensitive; never show child info to non-family users. - Store minimal PII; comply with local regulations (COPPA/GDPR depending on locality).

12. Operational: Background jobs, Monitoring, Scaling

Background jobs - Celery (Redis broker) or RQ for: thumbnail generation, expiry checks, invoice generation, reminders, analytics batch jobs.

Scaling strategy - Separate Web Servers for REST & Socket.IO for horizontal scale. - Use Redis for session/store and to coordinate socket rooms (if using multi-instance socket clusters). - Use Cloud SQL (managed Postgres), S3 (object storage), Cloud Redis.

Monitoring & Logs - Centralized logs: ELK / Datadog. - Transaction tracing: OpenTelemetry. - Alerts on failed background jobs and high error rates.

Backups - Regular DB snapshots. - S3 lifecycle rules for media.

13. UX & UI notes (wireframes + small details)

Important design principles - Family-first: show family name & quick child switcher. - Role-aware nav: hide payments/services appropriately. - Minimize cognitive load for parents: large buttons, confirmations for payments and refunds. - Child UI: fun, minimal text, big progress badges.

Wireframe snippets (text sketches)

- **Family Dashboard:** left column child switcher (avatars), main shows child cards + recent activity. Top-right notifications bell with unread badge.
- **Builder Card:** avatar, expertise tags, rating stars, cost chip, small gallery of images, CTA: "View profile" and "Compare" checkbox.
- **Builder Profile:** hero with gallery, services list (with "Book"), reviews, contact, average rating.

- **Assign Service Modal:** select child (card), select service, confirm price, select delivery mode (online/hybrid/offline), assign & create pending payment.
-

14. Roadmap & prioritized implementation plan

Phase 0 (MVP) - Auth, families, parents & children CRUD. - Builder profiles & services CRUD. - Subscriptions: request, approve (without complex scheduling), payment simulation, notification basic. - Media uploads (s3 presign) + gallery.

Phase 1 (Core) - Real payment gateway integration; idempotency keys. - Socket.IO notifications + persistent notification center. - Progress updates & timeline. - Scheduling & availability basics. - Refund & cancel flows with rules.

Phase 2 (Polish) - Catalog (ready projects), order flow. - Thumbnail generation, worker infra, expiry job. - Builder subscription tiers & promoted listings. - Analytics dashboard & admin features.

Phase 3 (Scale) - Escrow flows & automated payouts. - Background video processing & transcoding. - Globalization: multi-currency, localization. - Compliance & KYC at scale.

15. Appendix — critical code snippets

A. Transaction-safe subscription approval (SQLAlchemy + Flask)

```
# Approval snippet (already discussed in chat)
with db.session.begin():
    sub = db.session.execute(
        select(Subscription).where(Subscription.id ==
subscription_id).with_for_update()
    ).scalar_one_or_none()
    if not sub: raise NotFound
    if sub.payment_status == 'paid': raise Conflict
    # call payment gateway here (idempotent)
    sub.payment_status = 'paid'
    sub.status = 'active'
    db.session.add(sub)
# notify family via socket
```

B. Presigned S3 upload flow (backend)

```
import boto3
s3 = boto3.client('s3')

def presign_upload(filename, content_type, bucket, expires=3600):
    return s3.generate_presigned_post(
        Bucket=bucket,
        Key=filename,
        Fields={"Content-Type": content_type},
        Conditions=[["content-length-range", 1, MAX_BYTES]],
        ExpiresIn=expires,
    )
```

C. Socket join + notify (Flask-SocketIO)

```
@socketio.on('join_family')
def join_family(data):
    family_id = data.get('family_id')
    join_room(f'family_{family_id}')

def notify_and_save(family_id, payload):
    notif = Notification(family_id=family_id, message=payload['message'],
    data=payload)
    db.session.add(notif); db.session.commit()
    socketio.emit('notification', payload, room=f'family_{family_id}')
```

Final checklist to hand to developers (copy & paste)

1. Setup Postgres, Redis, S3. Add secrets via env.
 2. Create backend skeleton (services modules listed above).
 3. Build auth, user, builder flows.
 4. Implement subscription table + transaction-safe approve.
 5. Add WebSocket server and `notify_and_save`.
 6. Implement S3 presign + worker thumbnail generation.
 7. Add payments (sandbox / stripe/razorpay) with idempotency.
 8. Add front-end pages incrementally (dashboard, builder, catalog, subscriptions).
 9. Implement tests for race condition (two parents paying simultaneously).
 10. Deploy on staging; run expiry worker and test cancellation/refund notifications.
-

Need anything else?

If you'd like I can: - Export this into a prettier PDF or slides for sharing. - Generate uml/mermaid diagrams (if your canvas supports rendering them) and attach them. - Scaffold repository templates (backend+frontend starter code) with Dockerfiles and example env files.

You asked me to take all the time — I did my best to compile everything we discussed and expand it into a developer-ready spec.

If you want, I can now: - Generate mermaid diagrams inside this canvas for the architecture and a few workflows, or - Scaffold a starter repo (backend + frontend) with the folder structure and sample files.

Tell me which next step you prefer and I'll produce it right away.



Addendum — Reviews & Ratings (complete, production-ready)

1) Short summary (where it fits)

- Feature: Parents can rate & review builders for a specific service or completed project. Reviews may include text, star rating (1–5), optional images, and an optional `verified` badge if the parent actually paid for the service/order.
 - Builders can respond to reviews.
 - Admin/Builder moderation workflow to remove abusive reviews; spam detection and rate-limits.
 - Reviews affect builder ranking (search & sort), and the platform shows average rating (with smoothing) and review counts.
-

2) Database additions (Postgres SQL)

```
-- reviews table
CREATE TABLE reviews (
  id SERIAL PRIMARY KEY,
  parent_id INT REFERENCES parents(id),
  family_id INT REFERENCES families(id),
  builder_id INT REFERENCES builders(id),
  service_id INT REFERENCES services(id),           -- nullable: reviews for a
particular service
  project_order_id INT REFERENCES project_orders(id), -- nullable: review for
direct purchase
  subscription_id INT REFERENCES subscriptions(id),  -- nullable: review for
subscription
```

```

rating INT CHECK (rating BETWEEN 1 AND 5) NOT NULL,
title TEXT,
body TEXT,
images JSONB,          -- array of {url, thumbnail_url}
verified BOOLEAN DEFAULT FALSE, -- verified buyer flag (true if parent
purchased the service/order)
status TEXT DEFAULT 'visible', -- visible, flagged, removed
response TEXT,          -- builder reply (short)
response_at TIMESTAMP,
created_at TIMESTAMP DEFAULT now()
);

-- aggregate cache table (for fast reads)
CREATE TABLE builder_ratings_cache (
  builder_id INT PRIMARY KEY REFERENCES builders(id),
  avg_rating NUMERIC(3,2) DEFAULT 0,
  rating_count INT DEFAULT 0,
  display_rating NUMERIC(3,2) DEFAULT 0,
  updated_at TIMESTAMP DEFAULT now()
);

-- indexes for quick lookups
CREATE INDEX idx_reviews_builder ON reviews(builder_id, created_at DESC);
CREATE INDEX idx_reviews_service ON reviews(service_id);

```

Notes: - `verified` is set true automatically when review is created and DB finds an associated paid payment / project_order / subscription. - `builder_ratings_cache` stores aggregated metrics to avoid heavy `AVG()` on main table per-request.

3) Backend: API Endpoints (contract)

Public endpoints (parents & children view)

- `GET /builders/{builder_id}/reviews?page=1&pageSize=20&sort=recent|helpful|rating`
-> returns reviews (paginated) + `avg_rating` from cache + `rating_count`.
- `GET /services/{service_id}/reviews` — reviews for the specific service.

Parent actions (authenticated parent only)

- `POST /builders/{builder_id}/reviews`
Body:
`{ service_id?, subscription_id?, project_order_id?, rating: 1..5, title?, body?, images?: [s3_keys] }`

- Server verifies parent belongs to family; if parent paid for associated subscription/order, sets `verified=true`. On success:
- Insert into `reviews`.
- Update `builder_ratings_cache` (in a transaction).
- Push notification to builder via WebSocket: `{ type: "new_review", review_id, builder_id }`.
- Optionally notify admin if review contains flagged keywords or images.
- `PUT /reviews/{id}` — parent can edit review within a time window (e.g., 24–48 hours), only if they are the author and `status = visible` and not older than `X` days.
- `DELETE /reviews/{id}` — parent can delete their review (soft-delete via `status='removed_by_parent'`).

Builder actions

- `POST /reviews/{id}/response` — builder can post a short reply to a review (saved in `response` + `response_at`).
- `GET /builders/{builder_id}/reviews/pending_moderation` — admin/owner-only.

Admin actions

- `POST /admin/reviews/{id}/flag` — set `status='flagged'`.
- `POST /admin/reviews/{id}/remove` — set `status='removed'`.
- `GET /admin/reviews` — search/filter abusive reports.

4) Rating calculation best-practice (robust average)

- Use `builder_ratings_cache` updated transactionally on new review / edit / delete:
- $\text{rating_count} = \text{rating_count} + 1$
- $\text{avg_rating} = (\text{avg_rating} * (\text{rating_count} - 1) + \text{new_rating}) / \text{rating_count}$
- Also consider **Bayesian average** or Wilson score for better ranking when counts are low: compute a `display_rating` used for sorting:
- $\text{display_rating} = (C * m + \text{sum_ratings}) / (C + n)$ where `m` = global mean, `C` = confidence constant (e.g., 5), `n` = number of ratings for builder. This prevents new builders with a single 5★ review from ranking top.
- Cache `display_rating` alongside `avg_rating` in `builder_ratings_cache` to speed up search ordering.

5) Moderation & Trust Signals

- **Verified badge:** `verified` boolean in review if parent actually paid for the exact service/order (link check).

- **Automated spam filter:**
 - Block repeated identical reviews across many builders.
 - Flag suspicious content by keyword list (admin-review).
 - Limit images per review (e.g., max 3).
 - Rate-limit reviews per parent per time window.
- **Manual moderation:**
 - Admin panel shows flagged reviews and gives actions: warn builder/parent, remove review, ban builder, reply.
- **Builder responses:**
 - Show builder replies below the parent review; allow parents to comment back (one follow-up).
- **Helpfulness votes** (optional): other parents can upvote helpful reviews (affects sort by `helpful`). If you include it, add a `review_votes` table.

6) UI / UX (frontend components & behaviour)

Components

- `ReviewList` — paginated list with filters: `All | Verified | With Images | 5★ only`.
- `ReviewItem` — shows star rating (graphical), title, body, images thumbnails (click to enlarge), `verified` badge, timestamp, builder response, helpful button.
- `ReviewForm` — star selector (1-5), title input (optional), body textarea, image uploader (S3 presign), submit button. Show `You bought this` verified indicator in UI if verified.
- `ReviewSummary` — in builder header: average rating (big), rating count, breakdown histogram (5→1 stars).
- `ReviewAnalytics` — in builder dashboard: monthly average, reviews per month, sentiment trend.

Interactions

- After a parent leaves a review, show confirmation + a small animation; push real-time notification to builder.
- Display review images as lightbox.
- Show rating histogram with percentages.
- Sort options: `Most relevant (verified + helpful)`, `Most recent`, `Highest rating`, `Lowest rating`.

Where to place files (frontend)

Add components to `frontend/components/reviews/`:

```
frontend/components/reviews/
  ReviewList.tsx
  ReviewItem.tsx
  ReviewForm.tsx
  ReviewSummary.tsx
```

```
ReviewStats.tsx
review.css / tailwind classes
```

7) Backend files to add (service-level)

Add under `backend/services/builders/`:

```
backend/services/builders/
reviews_routes.py      # REST endpoints
reviews_crud.py        # DB operations (create, update cache, mark removed)
reviews_models.py      # SQLAlchemy models (if separate)
reviews_moderation.py  # automated checks / flagging logic
review_worker.py       # background worker to recompute caches / sentiment
```

8) Notifications & WebSocket hooks

- On new review: call `notify_and_save(family_id, payload)` or `notify_user(builder_id, payload)` to push real-time:
- Payload sample: `{ type: 'new_review', review_id, builder_id, parent_id, rating, verified }`.
- Push to builder room: `builder_{builder_id}` so their dashboard updates instantly.

9) Tests & QA

- Unit tests:
- Parent can create review only if authenticated and belongs to family.
- Verified flag true only when the parent has a paid order/subscription for that service/project.
- Average rating updated correctly after create/edit/delete.
- Race condition tests:
- Two parents attempting to review same subscription simultaneously — ensure both reviews can exist (reviews are independent), but verified flag logic does not conflict.
- Security tests:
- Ensure only parent who made purchase can mark review as verified (backend logic).
- E2E:
- Leave review with images → thumbnail generation → review shows images.

10) Analytics & Admin metrics

- Show builder dashboard:

- Average rating, monthly review count, recent flagged reviews, response rate.
- Admin panel:
- Reports for review abuse, top rated builders, suspicious accounts.

11) Additional small features / niceties (fully exhaustive)

- **Review helpfulness:** allow parents to mark a review as helpful; cache helpful count; use helpful count for sorting.
- **Review highlights:** auto-generate short highlights from long reviews (simple NLP) to show top pros/cons.
- **Reviewer profile:** show parent initials, first name, child grade (non-identifying) to add context.
- **Time-limited editing:** parents can edit their review for 48 hours after posting.
- **Review export:** allow builder to export their reviews (CSV) for record.
- **Auto-translation:** optional—translate reviews to user's language.
- **Sentiment analysis:** optional worker that stores `sentiment_score` to show trends.
- **Star breakdown microcopy:** show average rating (rounded to 1 decimal) and count (e.g., 4.6 • 124 ratings).

12) SQL migration snippet (example)

```
-- migration add reviews
CREATE TABLE reviews (
  id SERIAL PRIMARY KEY,
  parent_id INT REFERENCES parents(id),
  family_id INT REFERENCES families(id),
  builder_id INT REFERENCES builders(id),
  service_id INT REFERENCES services(id),
  subscription_id INT REFERENCES subscriptions(id),
  project_order_id INT REFERENCES project_orders(id),
  rating INT NOT NULL CHECK (rating BETWEEN 1 AND 5),
  title TEXT,
  body TEXT,
  images JSONB,
  verified BOOLEAN DEFAULT FALSE,
  status TEXT DEFAULT 'visible',
  response TEXT,
  response_at TIMESTAMP,
  created_at TIMESTAMP DEFAULT now()
);

CREATE TABLE builder_ratings_cache (
  builder_id INT PRIMARY KEY REFERENCES builders(id),
  avg_rating NUMERIC(3,2) DEFAULT 0,
  rating_count INT DEFAULT 0,
```

```
display_rating NUMERIC(3,2) DEFAULT 0,
updated_at TIMESTAMP DEFAULT now()
);

CREATE INDEX idx_reviews_builder ON reviews(builder_id, created_at DESC);
```

13) Where this plugs into the canvas content

- Add `Reviews & Ratings` subsection under: Feature List, DB schema, API surface, Frontend components, Backend files, Notifications, Tests, Admin.
- Update `builder profile` and `search` sections to use `builder_ratings_cache` for sorting.

14) UX copy & micro-interactions

- On review submit: "Thanks — your review is live. Verified badge appears if you bought this."
- If review is flagged/under review: show "This review is under review by our moderation team."
- Builder reply: "Builder replied — view reply" and show badge for response time (e.g., "Responded in 2 days").

15) Performance & caching considerations

- Don't compute `AVG(reviews.rating)` live for each builder page. Always read from `builder_ratings_cache`.
- Recompute `display_rating` on review create/update/delete. Use background worker to recompute if eventual consistency acceptable.
- Add DB constraint + caching invalidation for `builder_ratings_cache` updates.

16) Example Review JSON (API response)

```
{
  "id": 123,
  "parent_id": 45,
  "builder_id": 67,
  "rating": 5,
  "title": "Excellent!",
  "body": "Ramesh helped my child build the solar model in 2 sessions.",
  "images": [
    { "url": "https://s3/.../img1.jpg", "thumbnail": "https://s3/.../
thumb1.jpg" }
  ]
}
```

```
],  
  "verified": true,  
  "status": "visible",  
  "response": "Thanks! Glad Ananya enjoyed it. - Ramesh",  
  "created_at": "2025-08-20T10:02:00Z"  
}
```

17) Tests to add in test plan

- CRU(D) tests for reviews flow.
 - Verified-flag tests:
 - parent tries to review without paying -> `verified=false`.
 - parent pays then reviews -> `verified=true`.
 - Moderation tests:
 - flagged review becomes `status=flagged`.
 - admin remove -> `status=removed` and affects caches.
 - UI tests:
 - review form prevents more than 3 images, blocks >50MB videos.
 - review list paginates correctly.
-

Final note

All the above review & rating related points have been appended to the canvas document so your final PDF will include them. If you want, I can now regenerate a PDF of the canvas and provide it for download.